

Tarea - TIA-06 (Unidad 3 - Tarea 2)

Manipular bases de datos utilizando SGBD para la gestión de la información almacenada.

- **Tarea en Equipo**
- **Peso: 20%**
- **Práctica. Caso de Estudio**
- **DML - Manipulación de Datos**
- **Proyecto de Aula**

MIEMBROS DEL EQUIPO:

- Líder: Adel Angel Ramos
- Miembro: Juan José Rentería, Salome Murillo, Jerónimo Hoyos

Descripción de la Tarea

1. Contextualización

En el desarrollo del proyecto del sistema apícola, se ha identificado que uno de los principales riesgos en la construcción de sistemas de información no radica únicamente en la implementación técnica, sino en errores arrastrados desde las etapas iniciales del modelado conceptual y lógico. En este mismo orden de ideas, esos errores se pueden evidenciar en el modelo físico y afectar la manipulación de los datos y la construcción de consultas:

- Una base de datos física mal relacionada (referencias entre tablas)
- Sin controles para evitar inconsistencias y embasuramiento de datos

Estos problemas impactan directamente la calidad de la información, generando bases de datos:

- Vulnerables
- Inconsistentes y difíciles de mantener
- Ineficientes y de baja calidad
- Propensas a errores en consultas

Por esta razón, antes de iniciar la implementación física, es obligatorio realizar un proceso de: Revisión, corrección y mejora del modelo físico. Este proceso debe garantizar que la base de datos física sea:

- Coherente y correcta estructuralmente
- Protegida para evitar inconsistencias durante procesos de actualización
- Amigable y flexible para realizar consultas con información de calidad

En este contexto, el reto consiste en tomar el modelo físico previamente construido para el sistema apícola y transformarlo en un modelo completamente funcional, implementado en un SGBD, para poder realizar operaciones de inserción, modificación, eliminación y consultas a través del lenguaje de manipulación de datos (DML).

Es importante que al final se realicen pruebas para verificar el cumplimiento de las propiedades ACID (Véase Anexo A)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

2. Propósito de la Tarea

El objetivo de esta tarea es que los estudiantes:

- Detecten errores en su propio modelo físico y corregirlos. Justifiquen mejoras
- Realicen correctamente operaciones DML: INSERT, DELETE, UPDATE y SELECT
- Implementen operaciones DML en un SGBD (PostgreSQL en este caso)
- Implementen de vistas (VIEWS)
- Utilicen el relacionamiento entre tablas (JOIN), Funciones de agregación y consultas con parámetros.
- Verifiquen las propiedades ACID para establecer la calidad del modelo propuesto

3. Instrucciones de realización de la tarea

Experimentar en el SGBD establecido (PostgreSQL), de acuerdo con las necesidades de la red apícola. Luego, utilizando una herramienta gráfica (pgAdmin4), se enfocarán en la correcta manipulación del modelo, prestando especial atención al correcto funcionamiento de las instrucciones de inserción, actualización, eliminación y consulta. El proceso incluye una etapa de presentación y justificación de las decisiones de modelado, que permitirán la retroalimentación grupal guiada por el docente, para refinar el trabajo antes de la entrega final. Para evidenciar su aprendizaje, deberán organizar su trabajo en un repositorio de Git que contendrá varios entregables clave:

- Un informe técnico que justifique sus decisiones. Nota: Debe incluir las mejoras realizadas al Modelo Físico para permitir las consultas realizadas. Inventario de tablas definitivo
- El conjunto de scripts de manipulación solicitados en los formatos de plantilla suministrados.
- Análisis de los resultados, conclusiones generales y reflexiones individuales
- Un video corto donde sustenten el trabajo realizado y las conclusiones individuales que reflejen su aprendizaje personal. **DEBE MOSTRARSE EL ESTUDIANTE E IDENTIFICARSE CON SU NOMBRE. DEBE MOSTRAR CÓDIGO EN EJECUCIÓN** -> **Mostrar ejecución de scripts de creación, "poblamiento" y manipulación en el SGBD.**

Esta tarea les permitirá desarrollar competencias esenciales como el reconocimiento de sentencias DML y la implementación operaciones de manipulación de una base de datos, habilidades fundamentales para cualquier desarrollador de software. Adicionalmente, tendrá la oportunidad de realizar operaciones de auditoría a la base de datos para verificar su validez y calidad.

4. Criterios de entrega y valoración de la tarea

Los entregables finales para esta tarea, serán almacenados bajo la siguiente arquitectura: **Carpeta raíz: Tarea6. En esta carpeta estarán dos carpetas con los entregables: Informe, Resultados, Scripts y Video.**

Carpetas	Contenido requerido	Observaciones
/tarea6	Todas las subcarpetas y archivos de la tarea.	No aplica
tarea6/Informe	Informe detallado (plantilla suministrada)	.pdf
tarea6/Scripts	Script de creación de la base de datos Script de poblamiento de la base de datos Scripts de manipulación de la base de datos Scripts de verificación de propiedades ACID	.sql
tarea6/video	Video corto (5-10 minutos) con todos los miembros explicando una parte del trabajo y las conclusiones individuales.	Enlace a YouTube

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Estos son las plantillas de formato a utilizar OBLIGATORIAMENTE.
Solamente debe cambiar la “X” por el Equipo de estudiantes

	20261-bd1-g051-tia-04-equipox-informe.docx
	20261-bd1-g051-tia-04-equipox-resultados.xlsx
	20261-bd1-g051-tia-04-equipox-scripts-01-crea-postgresql.sql
	20261-bd1-g051-tia-04-equipox-scripts-01-modifica-01-postgresql.sql
	20261-bd1-g051-tia-04-equipox-scripts-01-modifica-02-postgresql.sql
	20261-bd1-g051-tia-04-equipox-scripts-02-crea-mysql.sql
	20261-bd1-g051-tia-04-equipox-scripts-03-crea-sqlserver.sql

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Evidencias de aprendizaje evaluadas

Evidencia	Habilidad evaluada
Informe con el paso a paso de la propuesta de solución, teniendo en cuenta: ● Corrección y presentación de las tablas definitivas de la Base de Datos ● Creación de la base de datos definitiva (CREATE) ● Poblamiento de las principales tablas indicadas (INSERT) ● Operaciones de actualización de la base de datos (UPDATE) ● Operaciones de eliminación de información de la base de datos (DELETE) ● Operaciones de consulta de la información de la base de datos (SELECT) ● Operaciones de consulta con vistas (VIEWS) ● Operaciones de consulta con parámetros. ● Operaciones de verificación de las propiedades ACID ● Utilización de JOIN, DISTINCT, GROUP BY, HAVING, Funciones de agregación (COUNT, SUM, MAX, MIN, AVG) ● El informe debe referenciar los scripts DML y explicar su estructura. ● Conclusiones individuales sobre el proceso de solución, destacando dificultades, dudas, aspectos relevantes del proceso.	Capacidad de análisis y redacción técnica Reflexión crítica y aprendizaje personal.
Tipos de dato SQL y NoSQL para big data e IoT con justificación técnica del uso de tipos de datos adecuados para escenarios de big data e IoT Ejemplo: campo JSON para las coordenadas geográficas u optimización del tamaño del dato para captar señales de sensores en IoT.	Comprensión y utilización de tipos de datos que representen una relación con soluciones big data e IoT.
Modelo Analítico implementado a través de operaciones DML: ● Inventario completo de tablas de la base de datos. ● Tablas con nombres según las reglas y la nomenclatura. ● Campos correctamente identificados con tamaño y tipo asociado. ● Relaciones adecuadas al caso (claves primarias y foráneas). ● Uso correcto las reglas, nomenclatura y estándares para BD físicas. ● Utilización de datos tipo JSON o JSONB en alguna Tabla PostgreSQL	Comprensión del Lenguaje de Manipulación de Datos (DML)
Participación en Foro de discusión y seguimiento de tareas	Comunicación asíncrona y trabajo en equipo
Video de sustentación (presentarse con imagen y nombre. Mostrar y explicar código en ejecución)	Comunicación oral y trabajo en equipo
Específicos	Competencia en gestión de versiones y organización digital.

ATENCIÓN: Analicen los criterios de evaluación y verifiquen su cumplimiento así como la ponderación de los ítems en la RÚBRICA.

BORRAR TODO LO ANTERIOR (DESDE AQUÍ) Y TODAS LAS INSTRUCCIONES EN AZUL Y ROJO DEL INFORME DE RESULTADOS ANTES DE ENTREGAR LA TAREA

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Informe con resultados

Equipo "5"

(TIA-6: Unidad 3 - Tarea 2)

1.- Inventario final de tablas del sistema de información

Cuadro. Inventario de Tablas definitivas de la Base de Datos

Nro	Tabla	Descripción	Tipo	Tablas Relacionadas
1	departamento	Departamentos de Colombia donde opera la red apícola	E	9
2	comunidad	Agrupaciones de apicultores con objetivos comunes	E	30
3	sensor	Sensores IoT instalados en los apiarios	E	17,24
4	entidad_financiera	Bancos y fondos que financian proyectos de investigación	E	11,33
5	entidad_reguladora	Organismos que emiten certificaciones sanitarias y de calidad	E	10
6	cooperativa	Cooperativas apícolas a las que puede pertenecer un apicultor	E	31,33
7	intercambio	Eventos de trueque de productos entre apicultores	E	12,29
8	producto	Productos apícolas: miel, cera, propóleo, jalea real, etc.	E	27,28
9	municipio	Municipios organizados por departamento por departamento	E	1, 13, 14, 15, 18
10	certificacion	Certificaciones emitidas por entidades reguladoras	E	5
11	investigacion	Proyectos de investigación apícola financiados	E	4,16
12	detalle_intercambio	Productos y cantidades que forman parte de un intercambio	R	7
13	apicultor	Productores apícolas registrados en la red	E	9, 18, 19, 20, 29, 30, 31, 32
14	consumidor	Clientes que realizan pedidos de productos apícolas	E	9,21
15	mercado	Mercado físico, virtual o híbrido, uno por municipio	E	9,21
16	evento	Eventos académicos o formativos ligados a investigaciones	E	11,32
17	historial_sensor	Historial de lecturas registradas por cada sensor IoT	R	3
18	apiario	Lugar físico donde el apicultor mantiene sus colmenas	E	13, 9, 22, 23, 24
19	gestion	Actividades de gestión realizadas por el apicultor	R	13
20	empleo	Publicaciones de ofertas de empleo creadas por apicultores	R	13
21	pedido	Cabecera del pedido realizado por un consumidor en un mercado	R	14,15,28
22	cosecha	Cosecha registrada en un apiario en una fecha determinada	R	18,25
23	registro_clinico	Historial sanitario y tratamientos aplicados en un apiario	R	18
24	medicion_ambiental	Lecturas de temperatura, humedad e IoT vinculadas a un apiario	R	3, 18, 26
25	lote	Lote de producción generado en una cosecha con costo registrado	R	22, 27, 28

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Nro	Tabla	Descripción	Tipo	Tablas Relacionadas
26	alerta	Alertas automáticas generadas por mediciones anómalas del sensor	R	24
27	lote_producto	Relación M:N entre lotes de producción y productos apícolas	R	25,8
28	detalle_pedido	Línea de pedido con producto, cantidad y precio histórico de venta	R	21,25,8
29	apicultor_intercambio	Participación M:N de apicultores en eventos de intercambio	R	13,7
30	apicultor_comunidad	Membresía M:N de un apicultor en una comunidad apícola	R	13,2
31	apicultor_cooperativa	Vinculación M:N de apicultor a cooperativa con fecha de ingreso	R	13,6
32	evento_apicultor	Asistencia M:N de apicultores a eventos de investigación	R	16,13
33	cooperativa_financiera	Relación M:N entre cooperativas y entidades financiadoras	R	6,4

Nota del grupo: La tabla apicultor_producto fue sustituida por la cadena lote_producto, lote , cosecha, apiario, apicultor. Esta decisión la consideramos mejor para el diseño pues que al implementar el apicultor_producto no iba a tener consistencia, lo que no le beneficiaba, entonces mantuvimos esta cadena ya que, para este diseño, le conviene más. También nos permite saber en qué cosecha se produjeron, en qué apiario y a qué costo, lo que habilita el cálculo de rentabilidad por productor. Una relación directa apicultor_producto hubiera perdido esa trazabilidad.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

2.- Script de “*creación*” de las tablas definitivas Bases de Datos (CREATE)

```
Query Query History
276     id_lote      INT NOT NULL,
277     id_producto INT NOT NULL,
278     PRIMARY KEY (id_lote, id_producto),
279     FOREIGN KEY (id_lote) REFERENCES lote(id_lote),
280     FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
281 );
282
283 CREATE TABLE detalle_pedido (
284     id_detalle      SERIAL      PRIMARY KEY,
285     id_pedido       INT         NOT NULL,
286     id_lote         INT         NOT NULL,
287     id_producto     INT         NOT NULL,
288     cantidad        INT         NOT NULL CHECK (cantidad > 0),
289     precio_unitario DECIMAL(10,2) NOT NULL CHECK (precio_unitario >= 0),
290     FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido),
291     FOREIGN KEY (id_lote) REFERENCES lote(id_lote),
292     FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
293 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 625 msec.

✓ Query returned successfully in 625 msec. ✕

3.- Script de “*poblamiento*” de la Bases de Datos (INSERT)

```
Query Query History
31 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (13, 3, 'Sogamoso');
32 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (14, 3, 'Chiquinquirá');
33 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (15, 3, 'Paipa');
34 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (16, 4, 'Bucaramanga');
35 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (17, 4, 'San Gil');
36 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (18, 4, 'Socorro');
37 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (19, 4, 'Barbosa');
38 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (20, 5, 'Neiva');
39 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (21, 5, 'Pitalito');
40 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (22, 5, 'Garzón');
41 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (23, 5, 'La Plata');
42 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (24, 6, 'Villavicencio');
43 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (25, 6, 'Granada');
44 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (26, 6, 'Acacías');
45 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (27, 7, 'Montería');
46 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (28, 7, 'Planeta Rica');
47 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (29, 7, 'Sahagún');
48 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (30, 8, 'Sincelejo');
49 INSERT INTO municipio (id_municipio, id_departamento, nombre) VALUES (31, 8, 'Corozal);
```

Data Output Messages Notifications

	id_departamento [PK] integer	nombre character varying (100)
6	6	Meta
7	7	Córdoba
8	8	Sucre
9	9	Magdalena
10	10	Valle del Cauca

Total rows: 10 Query complete 00:00:00.595 CRLF Ln 10265, Col 71

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

4.-Script de *actualización* de la información especificada (UPDATE)

The screenshot shows a SQL IDE interface with a dark theme. The main editor displays a script with the following SQL statements:

```
10 UPDATE mercado
11 SET descripcion = 'Reubicado - nueva direccion: Av. 19 #32-10, Zona Industrial'
12 WHERE id_mercado = 11;
13
14 UPDATE producto
15 SET precio = 27000.00
16 WHERE id_producto = 100;
17
18 UPDATE producto
19 SET precio = 22500.00
20 WHERE id_producto = 200;
21
22 UPDATE producto
23 SET precio = 38000.00
24 WHERE id_producto = 300;
25
26 SELECT id_producto, tipo_producto, precio FROM producto WHERE id_producto IN (100, 200, 300);
27 SELECT id_mercado, nombre, descripcion FROM mercado WHERE id_mercado IN (1, 6, 11);
28
```

Below the editor, the 'Data Output' tab is active, showing the results of the queries. The first query (SELECT) returned 3 rows, and the second query (SELECT) returned 3 rows. The results are displayed in a table format:

	id_producto	tipo_producto	precio
1	100	Producto	27000.00
2	200	Producto	22500.00
3	300	Producto	38000.00

	id_mercado	nombre	descripcion
1	1	Mercado Apícola de Cauca	Reubicado - nueva direccion: Cl 10 #5-20, Centro
2	6	Mercado Apícola de Fusagas...	Reubicado - nueva direccion: Cra 8 #12-45, El Carmen
3	11	Mercado Apícola de Tunja	Reubicado - nueva direccion: Av. 19 #32-10, Zona Indust...

A green status bar at the bottom right indicates: "Successfully run. Total query runtime: 162 msec. 3 rows affected."

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

5.-Script de *eliminación* de la información especificada (DELETE)

```
1  -- CASO 1: Insertar y eliminar un producto ingresado por error
2
3  INSERT INTO producto (id_producto, tipo_producto, descripcion, unidad, precio)
4  VALUES (700, 'Hidromiel', 'Bebida fermentada de miel - no se comercializara todavia', '750 ml', 45000.00);
5
6  SELECT id_producto, tipo_producto, precio FROM producto WHERE id_producto = 700;
7
8  DELETE FROM producto WHERE id_producto = 700;
9
10 SELECT id_producto, tipo_producto FROM producto WHERE id_producto = 700;
11
12 -- CASO 2: Insertar y eliminar un apicultor ingresado por error
13
14 INSERT INTO apicultor (id_apicultor, id_municipio, nombre, descripcion, telefono, correo, fecha_nacimiento)
15 VALUES (101, 1, 'Carlos Prueba Eliminar', 'Apicultor de prueba - no continuara en la red', '3199999999', 'prueba.e
16
17 SELECT id_apicultor, nombre, telefono FROM apicultor WHERE id_apicultor = 101;
18
```

Data Output Messages Notifications

id_apicultor	nombre
[PK] integer	character varying (80)

✓ Successfully run. Total query runtime: 135 msec. 0 rows affected. ✕

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

6.- Script de consultas de *listados* con la información especificada (SELECT**).**

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × Api-kconnect/postgres@PostgreSQL 18* ×

Api-kconnect/postgres@PostgreSQL 18

No limit

Query Query History Scratch Pad ×

```
3      p.fecha           AS fecha_pedido,
4      cons.id_consumidor,
5      cons.nombre       AS consumidor,
6      ap.id_apicultor    AS id_productor,
7      ap.nombre         AS productor,
8      pr.tipo_producto  AS producto,
9      dp.cantidad,
10     dp.precio_unitario AS precio_venta_cop
11 FROM pedido p
12 JOIN consumidor cons ON cons.id_consumidor = p.id_consumidor
13 JOIN mercado mer ON mer.id_mercado = p.id_mercado
14 JOIN detalle_pedido dp ON dp.id_pedido = p.id_pedido
15 JOIN lote l ON l.id_lote = dp.id_lote
16 JOIN cosecha c ON c.id_cosecha = l.id_cosecha
17 JOIN apiario api ON api.id_apiario = c.id_apiario
18 JOIN apicultor ap ON ap.id_apicultor = api.id_apicultor
19 JOIN producto pr ON pr.id_producto = dp.id_producto
20 WHERE mer.id_municipio = 5
21 ORDER BY p.fecha DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 159 Page No: 1 of 1

	id_pedido integer	fecha_pedido date	id_consumidor integer	consumidor character varying (80)	id_productor integer	productor character varying (80)	producto character varying (50)	cantidad integer	precio_venta_cop numeric (10,2)
1	73	2025-12-27	269	Daniela Pérez Díaz	13	Claudia Morales	Propóleo	2	37452.69
2	73	2025-12-27	269	Daniela Pérez Díaz	12	Felipe Castro	Jalea Real	1	42171.30
3	73	2025-12-27	269	Daniela Pérez Díaz	11	Patricia Vargas	Miel	2	27371.30
4	978	2025-12-26	202	Sebastián Jiménez Ri...	14	Roberto Jiménez	Miel	7	26161.82
5	51	2025-12-25	236	Raúl Gutiérrez Ruiz	10	Andrés Ramírez			

✓ Successfully run. Total query runtime: 130 msec. 159 rows affected. ✕

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

7.- Script de consultas de *grupos* con la información especificada (*GROUP BY* y *HAVING*)

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × Api-kconnect/postgres@PostgreSQL 18* ×

Api-kconnect/postgres@PostgreSQL 18

No limit

Query Query History Scratch Pad ×

```
1 SELECT
2     m.nombre                               AS municipio,
3     ap.ubicacion                           AS apiario,
4     COUNT(DISTINCT p.id_pedido)            AS total_pedidos,
5     SUM(dp.cantidad * dp.precio_unitario)  AS total_cop,
6     AVG(dp.precio_unitario)               AS precio_promedio_cop
7 FROM departamento d
8 JOIN municipio      m ON m.id_departamento = d.id_departamento
9 JOIN mercado        mer ON mer.id_municipio = m.id_municipio
10 JOIN pedido         p ON p.id_mercado      = mer.id_mercado
11 JOIN detalle_pedido dp ON dp.id_pedido     = p.id_pedido
12 JOIN lote           l ON l.id_lote         = dp.id_lote
13 JOIN cosecha        c ON c.id_cosecha      = l.id_cosecha
14 JOIN apiario        ap ON ap.id_apiario    = c.id_apiario
15 WHERE d.nombre = 'Antioquia'
16 GROUP BY m.nombre, ap.id_apiario, ap.ubicacion
17 HAVING SUM(dp.cantidad * dp.precio_unitario) > 500000
18 ORDER BY total_cop DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 24 Page No: 1 of 1

	municipio character varying (100)	apiario character varying (100)	total_pedidos bigint	total_cop numeric	precio_promedio_cop numeric
1	Santa Fe de Antioquia	Apiario El Paraiso - Santa Fe de Antioquia	31	55179622.41	311150.134411764706
2	Santa Fe de Antioquia	Apiario La Montaña - Santa Fe de Antioquia	32	32258332.06	174179.430540540541
3	Santa Fe de Antioquia	Apiario El Bosque - Santa Fe de Antioquia	23	27099620.89	240644.558148148148
4	El Bagre	Apiario Las Flores - El Bagre	17	19417070.24	115001.641200000000
5	Medellín (Santa Elena)	Apiario La Colina - Medellín (Santa Elena)	14	13992738.93	169816.8081

✓ Successfully run. Total query runtime: 111 msec. 24 rows affected. ✕

Total rows: 24 Query complete 00:00:00.111 CRLF Ln 18, Col 25

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

8.- Script de consultas con *vistas* con la información especificada (VIEW)

```
Query  Query History  Scratch Pad x
```

```
1  CREATE OR REPLACE VIEW vista_ventas_productor AS
2  SELECT
3      d.nombre                AS departamento,
4      m.nombre                AS municipio,
5      a.id_apicultor,
6      a.nombre                AS productor,
7      COUNT(DISTINCT p.id_pedido) AS total_pedidos,
8      SUM(dp.cantidad)          AS unidades_vendidas,
9      SUM(dp.cantidad * dp.precio_unitario) AS ingresos_cop,
10     AVG(dp.precio_unitario)    AS precio_promedio_cop,
11     MIN(dp.precio_unitario)    AS precio_min_cop,
12     MAX(dp.precio_unitario)    AS precio_max_cop
13 FROM apicultor a
14 JOIN municipio m ON m.id_municipio = a.id_municipio
15 JOIN departamento d ON d.id_departamento = m.id_departamento
16 JOIN apiario ap ON ap.id_apicultor = a.id_apicultor
17 JOIN cosecha c ON c.id_apiario = ap.id_apiario
18 JOIN lote l ON l.id_cosecha = c.id_cosecha
19 JOIN detalle_pedido dp ON dp.id_lote = l.id_lote
```

```
Data Output  Messages  Notifications
```

```
CREATE VIEW
Query returned successfully in 202 msec.
```

✓ Query returned successfully in 202 msec. ✕

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

9.- Script de consultas con *parámetros* con la información especificada (PREPARE**)**

```
1  -- PREPARE: consulta con 3 parametros, 3 JOIN, WHERE y HAVING
2  -- $1 = nombre del departamento (TEXT)
3  -- $2 = fecha minima del pedido (DATE)
4  -- $3 = monto minimo de ventas (NUMERIC)
5
6  PREPARE consulta_ventas_departamento (TEXT, DATE, NUMERIC) AS
7  SELECT
8      d.nombre                AS departamento,
9      m.nombre                AS municipio,
10     a.nombre                AS productor,
11     COUNT(DISTINCT p.id_pedido) AS total_pedidos,
12     SUM(dp.cantidad)         AS unidades_vendidas,
13     SUM(dp.cantidad * dp.precio_unitario) AS ingresos_cop,
14     AVG(dp.precio_unitario)  AS precio_promedio_cop
15 FROM apicultor a
16 JOIN municipio    m ON m.id_municipio = a.id_municipio
17 JOIN departamento d ON d.id_departamento = m.id_departamento
18 JOIN apiario      ap ON ap.id_apicultor = a.id_apicultor
19 JOIN cosecha       c ON c.id_apiario = ap.id_apiario
```

Data Output Messages Notifications

DEALLOCATE

Query returned successfully in 79 msec.

✓ Query returned successfully in 79 msec. ✕

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

10.- Script de verificación de *propiedades ACID*

The screenshot shows a PostgreSQL client interface with a dark theme. The top menu bar includes Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, and a connection tab for 'Api-kconnect/postgres@PostgreSQL 18*'. Below the menu is a toolbar with icons for file operations, query execution, and settings. The main area is divided into 'Query' and 'Query History' tabs. The 'Query' tab contains a SQL script with line numbers 18 to 36. The script performs several SELECT queries on 'producto' and 'mercado' tables, followed by a ROLLBACK and another set of SELECT queries. The 'Data Output' tab at the bottom shows the results of the last query, which is a SELECT statement filtering for 'id_mercado' in (1, 2). The output table has three columns: 'id_mercado' (integer), 'nombre' (character varying), and 'descripcion' (text). It contains two rows of data. A status bar at the bottom right indicates 'Successfully run. Total query runtime: 96 msec. 2 rows affected.'

```
18
19 SELECT id_producto, tipo_producto, precio
20 FROM producto
21 WHERE id_producto = 100;
22
23 SELECT id_mercado, nombre, descripcion
24 FROM mercado
25 WHERE id_mercado = 1;
26
27 ROLLBACK;
28
29 SELECT id_producto, tipo_producto, precio
30 FROM producto
31 WHERE id_producto IN (100, 200);
32
33 SELECT id_mercado, nombre, descripcion
34 FROM mercado
35 WHERE id_mercado IN (1, 2);
36
```

	id_mercado [PK] integer	nombre character varying (100)	descripcion text
1	1	Mercado Apícola de Cauc...	Reubicado - nueva direccion: Cll 10 #5-20, Ce...
2	2	Mercado Apícola de El Ba...	Punto de comercialización apícola en El Bagre

Showing rows: 1 to 2 | Page No: 1 of 1

✓ Successfully run. Total query runtime: 96 msec. 2 rows affected. ✕

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

11.- Script de demostración de inserción, actualización y consulta de una dato [JSON](#)

QueryQuery HistoryScratch Pad x

```
1  -- INSERCIÓN de un dato JSONB en la tabla apiario
2
3  INSERT INTO apiario (id_apiario, id_apicultor, id_municipio, ubicacion, ubicacion_geografica)
4  VALUES (
5      201,
6      1,
7      5,
8      'Apiario Vereda La Esperanza - Medellin',
9      '{
10         "coordenadas": {
11             "latitud": 6.284000,
12             "longitud": -75.57200
13         },
14         "altitud_msnm": 1720,
15         "region": {
16             "pais": "Colombia",
17             "departamento": "Antioquia",
18             "municipio": "Medellin",
19             "vereda": "La Esperanza"
20         }
21     }'
```

Data OutputMessagesNotifications

Showing rows: 1 to 1Page No: 1 of 1

	id_apiario [PK] integer	altitud_actualizada text
1	201	1850

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

12.- Análisis de resultados

Cuando empezamos a construir este proyecto teníamos claro el problema a resolver, pero a medida que avanzamos nos dimos cuenta de que el modelo conceptual con el que arrancamos tenía un error importante que el profesor señaló en la revisión: la base de datos podía registrar una venta completa, con su producto, su precio y su consumidor, pero era imposible saber quién la había producido. Teníamos la factura, pero no sabíamos de qué empresa venía.

Ese fue el cambio más grande que hicimos. El problema estaba en que `detalle_pedido` solo estaba conectado con producto, sin ningún camino hacia el apicultor. Para resolverlo incorporamos las tablas cosecha y lote como intermediarias entre la producción y la venta. El lote además guarda el costo de producción, lo que permite calcular cuánto gana o pierde cada productor. Con eso la base de datos dejó de ser un simple registro de ventas y se convirtió en una herramienta real de gestión económica para la red apícola.

También corregimos la relación entre los sensores IoT y los apiarios, porque en el modelo original una medición de temperatura no estaba vinculada al apiario donde se tomó, haciendo imposible responder qué colmena tuvo problemas. Se resolvió agregando la llave foránea del apiario directamente en `medicion_ambiental`.

En la implementación física aplicamos restricciones CHECK para proteger los datos desde la base, poblamos el sistema con 100 apicultores, 200 apiarios, 6 productos, 2000 consumidores y 2000 pedidos, y desarrollamos consultas que van desde listados simples hasta agrupaciones que responden cuánto gana cada productor y qué municipio mueve más dinero.

Ese recorrido, desde un modelo roto hasta una base de datos que responde preguntas de negocio reales, fue el aprendizaje más valioso de toda la tarea.

13.- Reflexiones y conclusiones individuales

Adel Ramos

Cuando empezamos este proyecto pensaba que bases de datos era solo crear tablas y meter datos. Pero el momento en que el profesor nos pregunto como sabiamos cuanto ganaba un productor y no supimos responder, me di cuenta que estabamos muy equivocados. Ese error en el modelo conceptual me enseño que no basta con que las tablas existan, tienen que estar bien conectadas para responder preguntas reales. Lo que mas me llevo de esta tarea es que diseñar bien una base de datos es mucho mas dificil de lo que parece, y que cada relacion que pones o que te falta tiene consecuencias reales.

Jeronimo Hoyos

Al inicio del semestre yo creía que esto iba a ser sencillo. Pero esta tarea me demostro lo contrario. Lo que más me costo fue entender los JOIN en cadena, como conectar apicultor con apiario, cosecha, lote y `detalle_pedido` para llegar hasta el pedido. Al principio me confundia con el orden pero cuando vi el resultado en pgAdmin ya todo tuvo sentido. Tambien aprendi que el PREPARE no es complicarlo todo sino hacer la consulta reutilizable, y eso en proyectos reales vale mucho. Me voy con mas conocimiento del que esperaba.

Salome Murillo

Antes de este semestre cuando escuchaba base de datos pensaba en una tabla de excel gigante y ya. Nunca imagine que detras de una aplicacion, un reporte o hasta el historial de compras de un supermercado habia todo un modelo diseñado con cuidado para que las preguntas importantes tuvieran respuesta.

Lo que mas me llevo es que en el mundo empresarial los datos sin estructura no valen nada. Una empresa puede tener millones de registros pero si no puede responder quien vende mas, que producto rota menos o donde estan sus clientes, esa informacion no sirve. Y eso depende completamente de que el modelo este bien hecho desde el principio. Aprendí que bases de datos no es solo una materia tecnica, es una habilidad de negocio.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Juan Jose Renteria

Yo creía que bases de datos era para los que iban a trabajar en tecnología pura. Pero este semestre me di cuenta que cualquier persona que trabaje con información, que hoy en día es casi todo el mundo, necesita entender esto.

Lo que más me sorprendió es que todo lo que se habla de inteligencia artificial y big data en las empresas depende de que los datos estén bien organizados por debajo. Si el modelo está mal, los reportes están mal, las decisiones están mal y el negocio sufre. Esta materia me mostró ese trabajo de fondo que nadie ve pero que sostiene todo lo demás, y ahora entiendo por qué es tan importante hacerlo bien.

15.- Video de Sustentación

The screenshot shows a video call interface with four participants in a 2x2 grid. The top-left participant is labeled 'ADEL ANGEL RAMOS CHAMORRO (Presentando y anotando)'. The top-right participant is 'Juan Jose Renteria'. The bottom-left participant is 'JERONIMO ...'. The bottom-right participant is 'SALOME MURILLO M...'. The main window displays the pgAdmin 4 interface. The 'Query' tab is active, showing a SQL script for creating a table named 'detalle_pedido'. The script includes constraints for primary and foreign keys. The 'Data Output' tab is empty, showing 'No data output. Execute a query to get output.' The bottom status bar shows the time '19:32' and the username 'hpx-whbk-pwv'.

16.- Repositorio GIT

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

adelramos424 / bd1-20261-g051-grupo5

Type (7) to search

+ 🔍 📄 📁 📧

<> Code 🔍 Issues 🔄 Pull requests 🛡️ Agents ⚙️ Actions 📁 Projects 📖 Wiki 🛡️ Security and quality 📊 Insights ⚙️ Settings

bd1-20261-g051-grupo5

Public

Pin

Watch 0

Fork 1

Star 0

main 1 Branch 0 Tags

Go to file T

Add file

<> Code

adelramos424 Delete tarea6/scripts/20261-bd1-g051-tia-06-equipos-5-scripts-11-json.sql c6932ae · 7 minutes ago 79 Commits

Tarea-01	Add files via upload	2 months ago
tarea4	Update link.txt	last month
tarea6	Delete tarea6/scripts/20261-bd1-g051-tia-06-equipos-5-scripts-11-json.sql	7 minutes ago
ADEL_RAMOS.png	Add files via upload	2 months ago
JERONIMO_HOYOS.png	Add files via upload	2 months ago
JUAN_JOSE_RENTERIA.png	Add files via upload	2 months ago
README.md	Fix formatting in team members section	2 months ago
SALOME_MURILLO.png	Add files via upload	2 months ago

README

About

Base de Datos Grupo 51, Grupo 5

Readme

Activity

0 stars

0 watching

1 fork

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

adelramos424

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Rúbrica: Criterios de Evaluación de la Tarea

#	Ítems Tarea	Peso	Cal
1	Inventario de tablas definitivo (corregido, si aplica)	10	
2	Script de creación de la base de datos (CREATE)	30	
3	Script de “ poblamiento ” de las tablas especificadas (INSERT)	80	
4	Script de actualización de la información especificada (UPDATE)	10	
5	Script de eliminación de la información especificada (DELETE)	10	
6	Script de consultas de listados con la información especificada (SELECT).	70	
7	Script de consultas de grupos con la información especificada (SELECT)	80	
8	Script de consultas con vistas con la información especificada (VIEW)	20	
9	Script de consultas con parámetros con la información especificada (PREPARE)	30	
10	Script de verificación de propiedades ACID	40	
11	Script de demostración de inserción, actualización y consulta de una dato JSON	10	
12	Análisis de resultados (en equipo - 300 palabras mínimo)	20	
13	Reflexiones y Conclusiones individuales (300 palabras mínimo)	30	
14	Informe: Uso de Plantillas, Estructura, Contenido y Calidad de presentación	40	
15	Video de Sustentación	150	
16	Repositorio GIT	20	
	NOTA = $xx/500 =$	Total	650
B	BONO ESPECIAL	RESPALDO PLANO DE LA BASE DE DATOS	100

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO A
Propiedades ACID

Propiedad	Descripción	Ejemplo
ATOMICIDAD (Atomicity)	Una transacción: ✓ se ejecuta completamente ✗ o no se ejecuta en absoluto <i>“O todo o nada”</i>	<pre>BEGIN; UPDATE cuenta SET saldo = saldo - 100 WHERE id = 1; UPDATE cuenta SET saldo = saldo + 100 WHERE id = 2; ROLLBACK;</pre> Resultado: no pasó nada
CONSISTENCIA (Consistency)	La base de datos siempre pasa de un estado válido a otro válido. - Se respetan reglas: - claves primarias - claves foráneas - CHECK - tipos de datos <i>“Datos válidos”</i>	<pre>INSERT INTO paciente (id_paciente) VALUES (NULL);</pre> Resultado: ✗ Falla si hay restricción → mantiene consistencia
AISLAMIENTO (Isolation)	Las transacciones no se “estorban” entre sí. - Cada usuario ve su propia transacción - Evita problemas como: - lecturas sucias - lecturas fantasma <i>“Transacciones independientes”</i>	Ejemplo conceptual - Usuario A actualiza datos - Usuario B no ve cambios hasta que A haga COMMIT
DURABILIDAD (Durability)	<i>“Si se guardó, se queda guardado”</i> <i>“Persistencia”</i> <i>“COMMIT no es solo guardar... es una promesa de que nunca se perderá.”</i>	<pre>BEGIN; INSERT INTO prueba_durabilidad (mensaje) VALUES ('Dato importante - no se debe perder');</pre> <pre>COMMIT;</pre> Resultado: ✓ Los datos sobreviven: - caídas del sistema - reinicios - fallos

“ACID es lo que evita que tu base de datos se convierta en caos.”

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO B

Cuadro Departamento-Municipios de Apiarios

Departamento	Municipio	Total Municipios	Productores	Apiarios	Consumidores
Antioquia	Caucasia	5	2	3	50
	El Bagre		2	3	50
	Zaragoza		2	3	50
	Santa Fe de Antioquia		3	5	50
	Medellín (Santa Elena)		5	10	100
Cundinamarca	Fusagasugá	5	3	5	50
	Girardot		2	5	50
	Ubaté		3	3	50
	Zipaquirá		3	5	50
	La Mesa		1	3	50
Boyacá	Tunja	5	4	10	50
	Duitama		3	3	50
	Sogamoso		2	5	50
	Chiquinquirá		1	3	50
	Paipa		1	3	50
Santander	Bucaramanga	4	4	8	50
	San Gil		1	3	50
	Socorro		2	3	50
	Barbosa		1	3	50
Huila	Neiva	4	4	10	50
	Pitalito		1	3	50
	Garzón		2	3	50
	La Plata		2	3	50
Meta	Villavicencio	3	4	9	50
	Granada		2	3	50

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

	Acacías		3	3	50
Córdoba	Montería	3	3	8	50
	Planeta Rica		1	3	50
	Sahagún		1	3	50
Sucre	Sincelejo	3	3	9	50
	Corozal		3	6	50
	Sampués		1	3	50
Magdalena	Santa Marta	3	4	9	50
	Ciénaga		3	5	50
	Fundación		3	5	50
Valle del Cauca	Cali	4	5	8	50
	Palmira		3	6	50
	Buga		3	7	50
	Tuluá		4	8	50
	TOTALES	37	100	200	2000

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO C

Cuadro de Productos con unidad de medida y costo aproximado (solo para ejercicio académico)

Código	Producto	Unidad típica	Precio aproximado (COP)
100	 Miel	1 kg	\$25.000
200	 Polen	250 g	\$20.000
300	 Propóleo	30 ml (extracto)	\$35.000
400	 Jalea Real	10–20 g	\$40.000
500	 Cera	1 kg	\$30.000
600	 Apitoxina	1 g	\$300.000

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO D

Especificaciones de Pedidos

1. Se está lanzando la Red Apícola apenas.
2. No todos los productores elaboran los 6 productos. Usted los asignará libremente.
3. Usted registrará 2000 pedidos libremente. Deben haber pedidos en todos los departamentos pero no necesariamente en todos los municipios.
 - Antioquia
 - Cundinamarca
 - Boyacá
 - Santander
 - Huila
 - Meta
 - Córdoba
 - Sucre
 - Magdalena
 - Valle del Cauca
4. Obligatoriamente, deben haber pedidos en los siguientes municipios:
 - Santa Fe de Antioquia
 - Medellín (Santa Elena)
 - Fusagasugá
 - Girardot
 - Tunja
 - Sogamoso
 - Bucaramanga
 - Neiva
 - Villavicencio
 - Monteria
 - Sincelejo
 - Corozal
 - Santa Marta
 - Ciénaga
 - Cali
 - Palmira
5. No necesariamente todos los consumidores harán pedidos
6. No necesariamente todos los productores recibirán pedidos
7. Los pedidos pueden tener uno o más tuplas en el detalle. Usted decide arbitrariamente. Cada tupla de detalle debe incluir el número de pedido, el código de producto, la cantidad y el precio (el precio puede cambiar eventualmente en la tabla productos y se debe conservar la información de venta original)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO E

Ejemplo de Tabla con información No Relacional
Tipos de Datos Semi-Estructurados (JSON/JSONB)

Tabla “apiario”

Campo: *ubicacion_geografica* JSONB

La ubicación no es solo latitud/longitud. En el mundo real incluye:

- Información jerárquica (departamento, municipio)
- Características del terreno
- Tipo de zona (rural/urbana/protegida)
- Metadatos adicionales

Esto hace que sea ideal como estructura semi-estructurada

Ejemplo de JSON

```
{
  "coordenadas": {
    "latitud": 6.25184,
    "longitud": -75.56359
  },
  "altitud_msnm": 1550,
  "region": {
    "pais": "Colombia",
    "departamento": "Antioquia",
    "municipio": "Medellín",
    "vereda": "Santa Elena"
  },
  "tipo_zona": "rural",
  "condiciones_ambientales": {
    "flora_dominante": ["eucalipto", "café"],
    "fuente_agua_cercana": true
  }
}
```

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO F

Ejemplo de inserción y consulta de datos en la Tabla “apiario”

Inserción de información

```
INSERT INTO apiario (nombre, ubicacion_geografica)
VALUES
(
    'Apiario La Montaña',
    '{
        "coordenadas": {
            "latitud": 6.25184,
            "longitud": -75.56359
        },
        "altitud_msnm": 1550,
        "region": {
            "pais": "Colombia",
            "departamento": "Antioquia",
            "municipio": "Medellín",
            "vereda": "Santa Elena"
        },
        "tipo_zona": "rural",
        "condiciones_ambientales": {
            "flora_dominante": ["eucalipto", "café"],
            "fuente_agua_cercana": true
        }
    }'
),
(
    'Apiario El Bosque',
    '{
        "coordenadas": {
            "latitud": 4.7110,
            "longitud": -74.0721
        },
        "altitud_msnm": 2600,
        "region": {
            "pais": "Colombia",
            "departamento": "Cundinamarca",
            "municipio": "Bogotá",
            "vereda": "Usme"
        },
        "tipo_zona": "periurbana",
        "condiciones_ambientales": {
            "flora_dominante": ["pino", "acacia"],
            "fuente_agua_cercana": false
        }
    }'
);
```

Consultas de verificación

```
SELECT * FROM apiario
WHERE ubicacion_geografica->'region'->>'departamento' = 'Antioquia';

SELECT * FROM apiario
WHERE (ubicacion_geografica->>'altitud_msnm')::int > 1500;
```